

Rotman

INTRO TO DATA VISUALIZATION

Part III Plotting Maps (with GeoPandas & Matplotlib) – Reproducing the Toronto Parking Tickets Visualization

September 15, 2026 Prepared by Jay Cao / [MDAL](#)

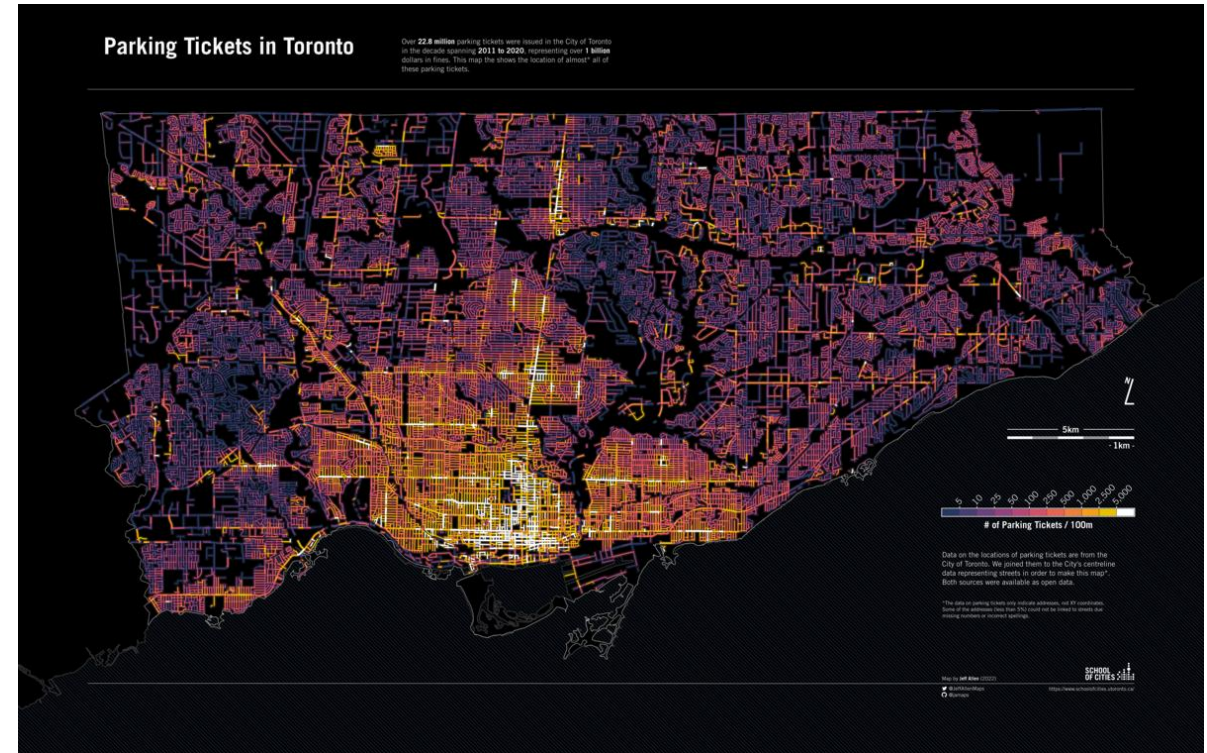
Website: <https://rmdal.github.io/mma-dv-2026/>



Rotman School of Management
UNIVERSITY OF TORONTO

Walk-through – Toronto Parking Ticket

- Reproduce (almost) Jeff Alan's [Toronto Parking Tickets](#) Visualization
- I believe Jeff's plot is done using QGIS and Inkscape
- We will use GeoPandas with Matplotlib



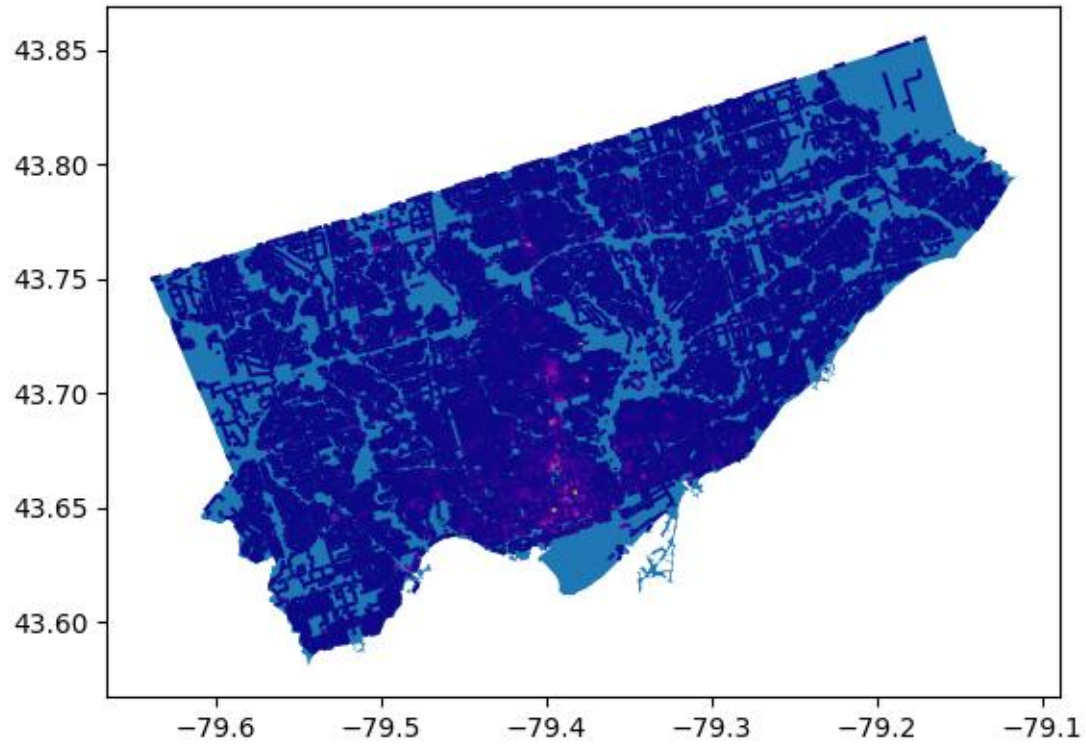
Parking Tickets in Toronto by Jeff Alan

Ref: 1) <https://schoolofcities.github.io/parking-tickets-toronto/>
2) <https://github.com/schoolofcities/parking-tickets-toronto/tree/main>

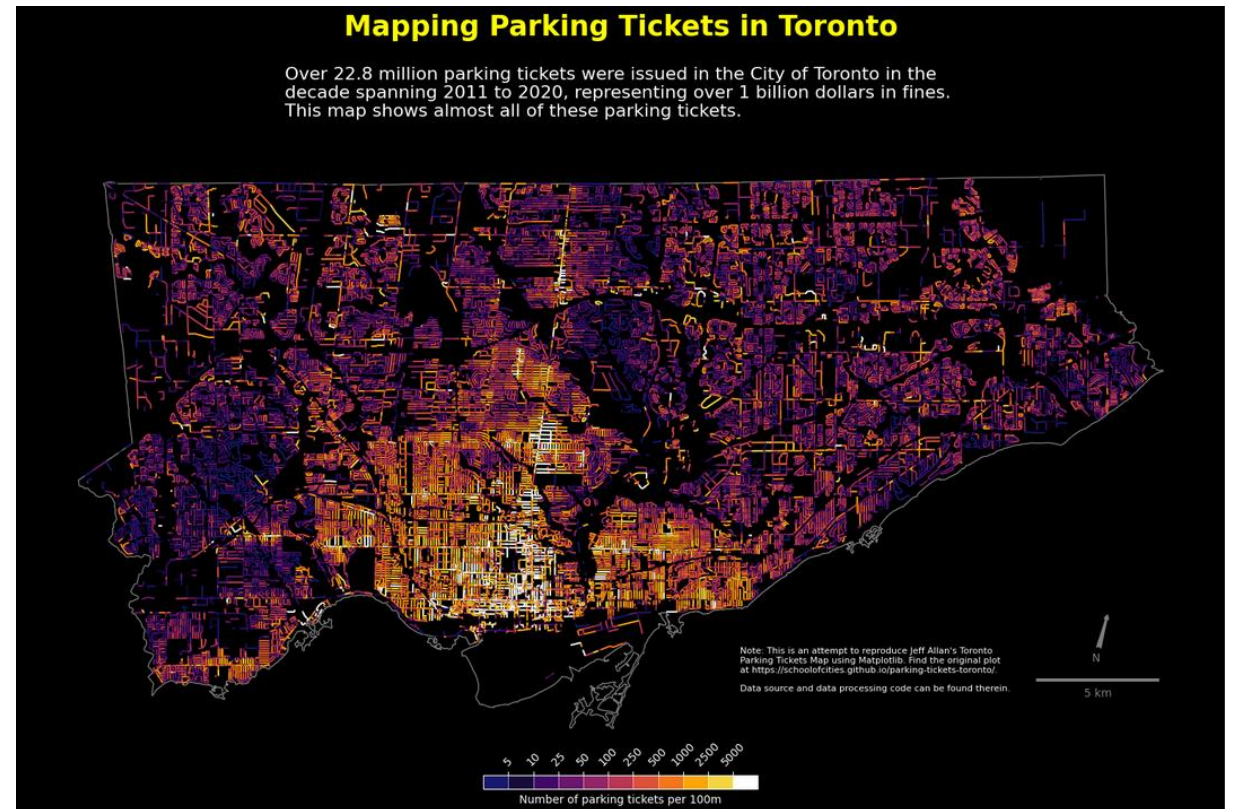
Datasets

- Toronto parking ticket count dataset
 - non-identifiable information relating to each parking ticket in Toronto
 - [A cleaned-up version](#) at the original plot git repo
- Toronto boundary dataset
 - the geographical administrative boundary of the City of Toronto
- Toronto Centreline dataset
 - a data set of linear features representing streets/roads, walkways, rivers, railways, highways and administrative boundaries within the City of Toronto.

Walk-through – Our Implementation



the default plot from Geopandas/Matplotlib



the refined plot

Default Plot

...

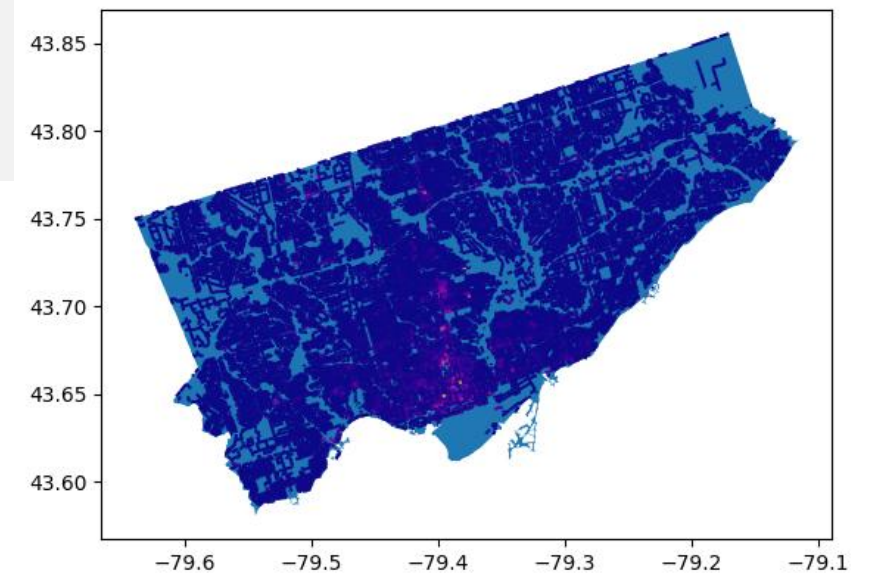
```
# draft a heatmap plot  
fig, ax = plt.subplots()
```

```
boundary.plot(ax=ax)  
centreline.plot(ax=ax, column='count_all', cmap='plasma')
```

...

```
centreline.head()
```

	CENTRELINE_ID	FEATURE_CODE_DESC	geometry	count_all
0	914600	Local	MULTILINESTRING ((-79.50875 43.59744, -79.5098...	8.0
1	914601	Local	MULTILINESTRING ((-79.50987 43.59720, -79.5103...	355.0
2	7862398	Local	MULTILINESTRING ((-79.51087 43.59697, -79.5113...	708.0
3	914587	Major Arterial	MULTILINESTRING ((-79.51805 43.59795, -79.5191...	205.0
4	6735911	Major Arterial	MULTILINESTRING ((-79.51914 43.59770, -79.5202...	325.0



- Ref: 1) <https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.plot.html>
2) <https://matplotlib.org/stable/users/explain/colors/colormaps.html>

Refine 1. Better Orientation

```
# rotate centreline
# turn EPSG:4326 to EPSG:3347 first to avoid shape
# distortion after rotation
cline_3347 = centreline.to_crs(epsg=3347)

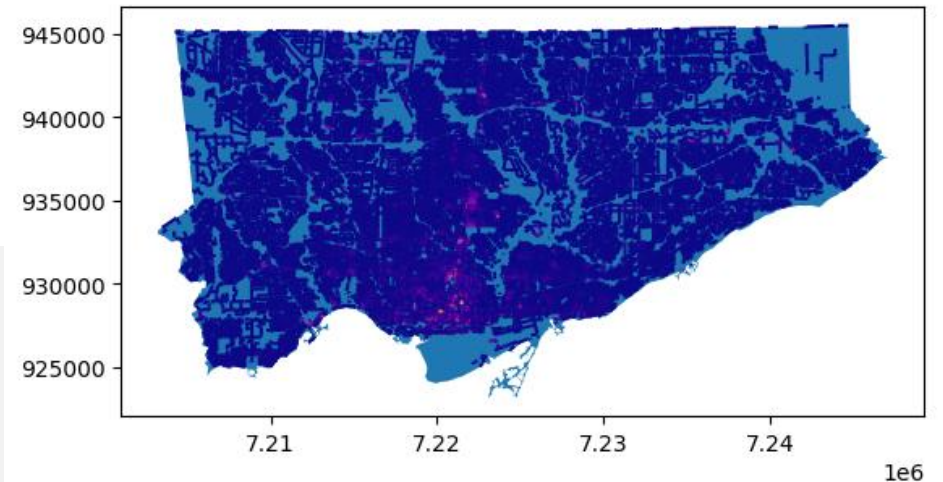
# rotate with respect to the centroid of all centrelines
cline_3347_rotated = cline_3347.rotate(-28, origin=cline_3347.union_all().centroid)
    .rename("geometry_3347_rotate")

# combine the original centreline GeoDataFrame with the rotated GeoSeries
centreline_rotated = centreline.join(cline_3347_rotated)

...

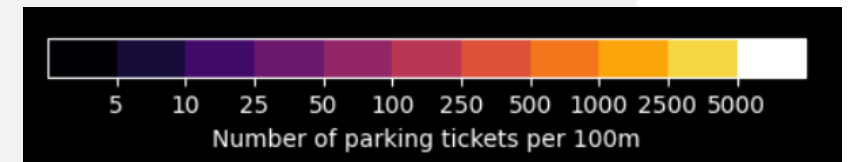
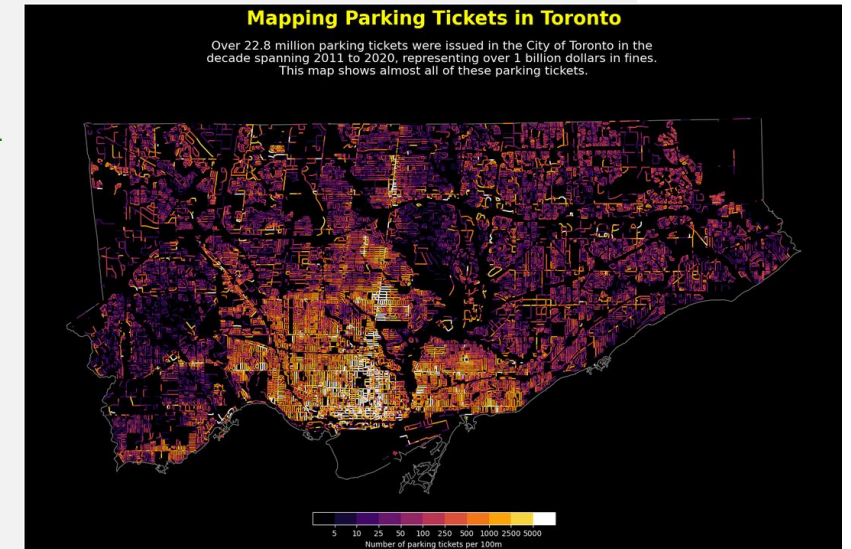
# rotate boundary
# turn EPSG:4326 to EPSG:3347 first to avoid shape distortion after rotation
# rotate with respect to the centroid of all centrelines to match centreline rotation centroid
boundary_rotated = boundary.to_crs(epsg=3347).rotate(-28, origin=cline_3347.union_all().centroid)

...
```



Refine 2. Colormap on Discrete Intervals

```
...  
  
# Generate a colormap index based on discrete intervals  
# https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.Colormap.html  
# https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.BoundaryNorm.html  
cmap = plt.colormaps['inferno'].with_extremes(over="white")  
bounds = [5, 10, 25, 50, 100, 250, 500, 1000, 2500, 5000]  
norm = BoundaryNorm(bounds, cmap.N, extend='both')  
  
# plot centreline heatmap  
centreline_rotated.plot(ax=ax, column='count_all',  
                        cmap=cmap,  
                        norm=norm,  
                        markersize=0.5,  
                        legend=True,  
                        legend_kwds={  
                            'shrink': 0.3,  
                            'orientation': 'horizontal',  
                            'pad': 0,  
                            'anchor': (0.5, 1),  
                            'extendfrac': 'auto',  
                            'extendrect': True,  
                            'label': 'Number of parking tickets per 100m'})
```



Ref: 1) [Discrete and extended colorbar with continuous colorscale](#)

2) In general, you can use [mapclassify](#) to auto-generate classification schemes for choropleth maps.

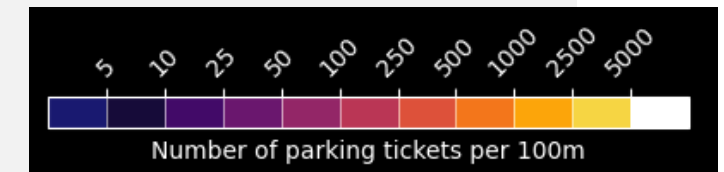
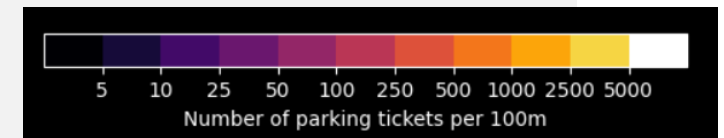
Refine 3. Better Colorbar

...

```
# plot colorbar legend separately so as to customize its look
cbar = fig.colorbar(ScalarMappable(norm=norm, cmap=cmap),
                    ax=ax,
                    orientation='horizontal',
                    shrink=0.3,
                    pad=-0.02,
                    anchor=(0.5, 1),
                    extendfrac='auto',
                    extendrect=True,
                    drawedges=True,
                    label='Number of parking tickets per 100m')

cbar.ax.tick_params('x',
                    bottom=False, labelbottom=False,
                    top=True, labeltop=True,
                    labelrotation=45)
```

...



Refine 4. Map Scale, North Arrow, & Notes

```
# add scalebar
# https://geopandas.org/en/stable/gallery/matplotlib_scalebar.html
scale = ScaleBar(dx=1,
                 location='lower right',
                 color='grey',
                 box_alpha=0,
                 width_fraction=0.005,
                 border_pad=5)

_ = ax.add_artist(scale)

# add north arrow
# https://matplotlib.org/stable/users/explain/text/annotations.html
_ = ax.annotate("N",
                xy=(0.91, 0.25), xycoords='figure fraction',
                xytext=(0.9, 0.19), textcoords='figure fraction',
                ha='center',
                color='gray',
                arrowprops=dict(arrowstyle="fancy", color="gray"))

# add notes
_ = ax.text(0.6, 0.11,
           ("Note: This is an attempt to reproduce Jeff Allan's Toronto \nParking Tickets Map using Matplotlib. "
            "Find the original plot \nat https://schoolofcities.github.io/parking-tickets-toronto/.\n\n"
            "Data source and data processing code can be found therein."),
           transform=ax.transAxes,
           wrap=True,
           fontsize=8,
           horizontalalignment='left',
           bbox=dict(boxstyle='square', pad=1, facecolor='black', edgecolor='black'))
```

...

